



Smart Hospital Bed System

Complete Project Documentation

Architecture, usage, deployment and modification guide

Flutter (web + Android) | FastAPI | MongoDB Atlas

Version 1.0 | October 2026

Right bed. Right hospital. Right now.

Contents

1. What this project is	3
2. Key features	3
3. Users and permissions	4
4. System architecture	4
5. Technology stack	5
6. Project structure	6
7. Database design	6
8. Core logic explained	8
9. API reference	10
10. The Flutter app	11
11. Demo data and logins	12
12. Running the project locally	13
13. Configuration and environment variables	14
14. Deployment guide	14
15. Security design	15
16. Testing and quality checks	15
17. How to modify and extend	15
18. Troubleshooting	17
19. Limitations	17
20. Glossary	18

1. What this project is

Smart Hospital Bed System is a full-stack web and mobile application that shows, in real time, which hospitals have the beds and equipment a patient needs, and lets a patient or ambulance coordinator send a referral request that the hospital can accept or reject. When a hospital accepts, its capacity numbers update automatically.

1.1 The problem it solves

In an emergency, families and ambulance crews call hospital after hospital asking whether an ICU bed, a ventilator or an emergency bed is free. This wastes critical minutes and the answers are often out of date. There is no shared, live picture of capacity across a city.

1.2 The solution in one loop

The whole product is one loop. Everything else in the project supports it:

Hospital updates its beds -> patient searches by need and location -> system filters and ranks hospitals -> patient sends a request -> hospital accepts -> bed count drops.

The system is deliberately not a hospital directory. It filters out hospitals that cannot meet a hard requirement (for example, an ICU bed is free but there is no ventilator), then ranks the rest by a match percentage.

1.3 Is it a website or an app?

Both. It is one Flutter codebase that builds into a **web app** (opens in any browser at a public link) and an **Android app** (APK). Both talk to the same backend server and database.

1.4 Demo region

The seed data is set in **Hyderabad, Sindh, Pakistan**. Hospital names are fictional on purpose so that no made-up bed numbers are attached to real institutions. The code is region-agnostic: changing the seed data changes the city (see section 17).

2. Key features

- **Live capacity management:** hospital staff update total, occupied and temporarily unavailable counts for ten resource types. Available capacity is calculated automatically.
- **Smart matching:** hard filter on required resources, then a weighted score (distance, availability, low occupancy) shown as a match percentage. Unsuitable hospitals are listed with the reason.
- **Referral workflow:** request, review, accept or reject, patient transfer, admission. Statuses are tracked and visible to both sides.
- **Reservation control:** a bed is held for 15 minutes when a request is sent. Atomic database updates prevent two people from getting the same last bed. Expired holds are released automatically.
- **Role-based access:** patient, coordinator, hospital staff and administrator each see different screens and can call different endpoints.
- **Dashboards and analytics:** hospital-level and system-level charts (occupancy, requests by status, response time, daily trends).
- **Data trust:** last-updated timestamps, a stale-data warning after 30 minutes, and an audit log of every capacity change.
- **Map system:** hospitals on an OpenStreetMap map, user location, and one-tap navigation in Google Maps.

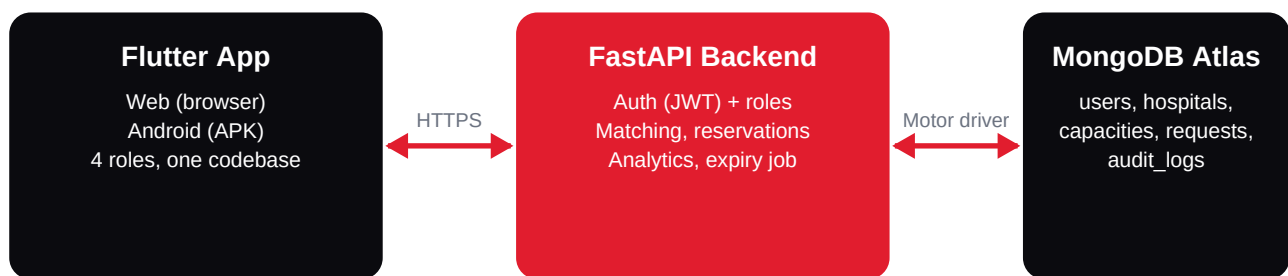
- **Admin tools:** verify or reject hospitals, spot outdated records, view all hospitals on a map.

3. Users and permissions

Role	What they can do	Main screens
Patient / attendant	Search hospitals by needed resources and location, compare results, send a referral request, track its status, cancel a pending request, navigate to the hospital.	Find Hospital, My Requests
Ambulance / emergency coordinator	Same as patient, framed as comparing hospitals and tracking several referrals.	Compare Hospitals, My Requests
Hospital staff	Update capacity for their own hospital only, see incoming requests, accept or reject, move accepted patients to Transferred and Admitted, view their dashboard.	Capacity, Requests, Dashboard
Administrator	Verify or reject hospitals, view system-wide analytics, find hospitals with outdated data, view all hospitals on a map.	System Analytics, Hospitals, Map

Permissions are enforced on the server (JWT role check), not only by hiding buttons in the app.

4. System architecture



The app never talks to the database directly. Credentials stay on the server.

4.1 The three layers

- **Flutter app (frontend).** Draws the screens, holds the logged-in session, calls the backend with Dio and polls for live updates every few seconds.
- **FastAPI backend.** The only component that touches the database. Handles login, role checks, validation, search and matching, reservations, analytics and the background expiry job.
- **MongoDB Atlas (database).** Stores users, hospitals, capacity records, requests and audit logs. Uses a 2dsphere geospatial index for nearest-hospital queries.

4.2 Why there is a backend between Flutter and MongoDB

Flutter code runs on the user's device or in their browser, so anything inside it can be extracted. If the app connected directly to MongoDB, the database password would be exposed and anyone could read or change any record. The backend keeps credentials private, enforces who can do what, and runs the sensitive logic (matching and reservations) in one trusted place.

4.3 How live updates work

MongoDB does not push changes to the client by itself. The app uses short polling instead: the staff Requests tab and the patient My Requests tab refresh every 4 seconds, and dashboards every 10 seconds. For a hospital demo this feels live and is much simpler than WebSockets.

4.4 End-to-end example

- A staff member at Hirabad Medical Centre sets ICU occupied to 7. The backend recalculates available, stamps last_updated and writes an audit log.
- A patient searches for ICU + ventilator. The backend uses a geospatial query to find nearby verified hospitals, joins their capacity records, filters, scores and returns a ranked list plus an excluded list.
- The patient taps Send Request. The backend atomically reserves one ICU bed and one ventilator and sets a 15-minute expiry.
- The hospital's Requests tab shows the request within 4 seconds. Staff tap Accept. Reserved becomes occupied.
- The patient's tracker moves to Accepted and offers a Navigate button that opens Google Maps.

5. Technology stack

Layer	Technology	Why it was chosen
Frontend	Flutter (Dart), Material 3	One codebase for web and Android.
State management	provider	Simple and enough for session and screen state.
HTTP client	dio	Interceptors attach the JWT to every request and handle errors centrally.
Charts	fl_chart	Bar, pie and line charts for dashboards.
Maps	flutter_map + latlong2 (OpenStreetMap tiles)	Free, no API key, no billing.
Location and links	geolocator, url_launcher	GPS location and opening Google Maps for directions.
Fonts	google_fonts (Inter, Sora)	Modern look for the redesigned UI.
Backend	Python, FastAPI, Uvicorn	Fast to build, automatic API docs at /docs.
Database driver	motor (async MongoDB)	Non-blocking database access for FastAPI.
Auth	python-jose (JWT), bcrypt	Stateless tokens with roles; salted password hashing.
Validation	Pydantic	Typed request and response models.
Database	MongoDB Atlas (free tier)	2dsphere geo queries, atomic find_one_and_update, flexible schema.
Hosting (planned)	Render or Railway (backend), Vercel or Firebase Hosting (web)	Free tiers, simple deploys.

5.1 Why MongoDB instead of Supabase

The project was built by AI-assisted coding. MongoDB needs only one connection string, while Supabase needs SQL scripts, row-level security policies and several keys, which is where AI-assisted builds often break. MongoDB also gives geospatial search, atomic counters for bed reservations, and a flexible schema for the many resource types.

6. Project structure

File names below follow the build plan. If your repository differs slightly, the purpose of each folder is the same.

6.1 Repository layout

smart-hospital/	
README.md	project overview, run steps, demo logins
backend/	FastAPI server
main.py	creates the app, CORS, registers routers, starts expiry job
database.py	MongoDB connection and index creation on startup
models/	Pydantic models: User, Hospital, Capacity, Request, AuditLog
routes/	HTTP endpoints, grouped by feature
auth.py	login, current user
capacity.py	capacity read and update
search.py	hospital search and matching
requests.py	referral requests and status changes
analytics.py	hospital and system analytics, admin hospital tools
services/	business logic used by the routes
auth.py	password hashing, JWT, role dependency
capacity.py	capacity validation, recalculation, audit log
matcher.py	hard filter and scoring
referral.py	atomic reservation, accept, reject, cancel, expiry
analytics.py	MongoDB aggregation pipelines
seed.py	hospitals, capacities and demo users
seed_requests.py	about 40 fake past requests for charts
reset_demo.py	re-runs both seed scripts for a clean demo
requirements.txt	Python dependencies
.env	secrets (never committed)
.env.example	variable names without values
frontend/	Flutter app
pubspec.yaml	dependencies
lib/	
main.dart	app entry, providers, routing by role
config/	app_config.dart (API URL), app_theme.dart (colors, fonts)
models/	Capacity, ReferralRequest, hospital and search result models
services/	api_client.dart, staff, patient, admin and map services
providers/	auth_provider.dart (login, session restore)
screens/	
auth/	login screen
patient/	find hospital, results, my requests
staff/	capacity, requests, dashboard
admin/	system analytics, hospitals, map
widgets/	AppBar, AppCard, StatTile, StatusChip, MatchRing, hospital_map ...

6.2 How the pieces connect

- A screen calls a method in a **service** (frontend), which uses the **api_client** to call an endpoint.
- The endpoint lives in **routes/**. It checks the JWT and role, validates the body, then calls a function in **services/** (backend).
- The backend service reads or writes MongoDB through **database.py** and returns JSON.
- The screen shows the result through a **provider** or local state, using shared **widgets**.

7. Database design

MongoDB database with five collections. Documents are shown as field lists.

7.1 users

Field	Type	Notes
email	string	Unique index. Used to log in.
password_hash	string	bcrypt hash. Never returned by the API.
name	string	
role	string	patient, coordinator, staff or admin.
hospital_id	ObjectId	Only for staff. Links the user to one hospital.

7.2 hospitals

Field	Type	Notes
name, address, contact	string	Display details.
location	GeoJSON Point	{type: Point, coordinates: [longitude, latitude]}. Note the order: longitude first. Has a 2dsphere index.
verification_status	string	pending, verified or rejected. Only verified hospitals appear in search.
account_status	string	active or inactive. Only active hospitals appear in search.
emergency_available	bool	

7.3 capacities

One document per hospital per resource type. A unique compound index on (hospital_id, resource_type) prevents duplicates.

Field	Meaning
hospital_id, resource_type	Which hospital and which resource.
total	Total capacity of this resource.
occupied	In use by admitted patients.
reserved	Temporarily held for pending requests (15-minute holds).
temporarily_unavailable	Out of service (cleaning, maintenance, broken equipment).
available	Calculated: total - occupied - reserved - temporarily_unavailable.
last_updated	UTC timestamp of the last change. Drives the stale-data warning.

Resource types: general_bed, emergency_bed, icu_bed, nicu_bed, ventilator, operation_theatre, isolation_bed, dialysis, trauma, ambulance.

7.4 requests

Field	Meaning
patient_reference	Name or reference typed by the requester. Avoid real personal data in demos.
requester_id	The user who sent the request.
required_resources	List of resource types needed.
urgency	low, medium, high or critical.
location	GeoJSON Point of the patient. Staff can open it in Google Maps.
selected_hospital_id	The hospital the request was sent to.
status	request_sent, hospital_reviewing, accepted, patient_transferred, admitted, rejected, cancelled, expired or no_capacity.
created_at, accepted_at	Used to compute average response time.
reservation_expires_at	created_at plus 15 minutes.

7.5 audit_logs

Every capacity change writes one entry: user_id, hospital_id, resource_type, old_values, new_values and timestamp. This allows an admin to review suspicious updates.

7.6 Indexes created on startup

- hospitals.location: 2dsphere (nearest-hospital queries).
- users.email: unique.
- capacities (hospital_id, resource_type): unique compound.
- requests.status: regular index for fast filtering.

8. Core logic explained

8.1 Capacity calculation

```
available = total - occupied - reserved - temporarily_unavailable
```

The backend rejects any update that makes a number negative or makes occupied + reserved + temporarily_unavailable exceed total (HTTP 400 with a clear message). Staff can set only total, occupied and temporarily_unavailable. The reserved count is controlled by the referral system.

8.2 Search and smart matching

The search endpoint works in four stages:

- **Geo query.** A MongoDB \$geoNear aggregation returns verified, active hospitals within the maximum distance, with the distance in km.
- **Join.** Each hospital's capacity records for the required resources are attached.
- **Hard filter.** A hospital is suitable only if every required resource has available of at least 1. Others go to an excluded list with a reason such as 'No ventilator available'.
- **Score.** Suitable hospitals get a match percentage and are sorted from highest to lowest.

```
score = 0.5 * distance_score + 0.3 * availability_score + 0.2 * low_occupancy_score
```

```
distance_score      = max(0, 1 - distance_km / max_distance_km)
availability_score   = average over required resources of (available / total)
```

```

low_occupancy_score    = 1 - average over required resources of (occupied / total)

match_percent          = round(score * 100)
estimated_travel_min    = distance_km / 30 * 60      (assumes 30 km/h city speed)

```

Because filtering happens first, the closest hospital is never recommended if it cannot meet a hard requirement. The seed data includes two deliberate examples: Hirabad Medical Centre (ICU free, zero ventilators) and Saddar City Hospital (ICU completely full). Both appear under 'Not suitable'.

8.3 Reservations and double-booking protection

When a request is sent, each required resource is reserved with one atomic MongoDB operation:

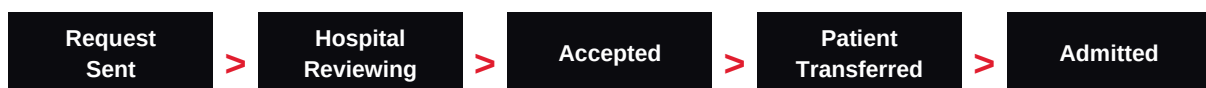
```

find_one_and_update(
  filter = { hospital_id, resource_type, available: { $gte: 1 } },
  update = { $inc: { available: -1, reserved: +1 }, $set: { last_updated: now } }
)

```

The filter and the update happen together inside the database, so if two people tap Send at the same moment for the last bed, only one operation matches. The other gets no document back and the API returns HTTP 409 (No capacity available). If a multi-resource request fails on its second resource, the first reservation is rolled back so no reserved count is leaked.

8.4 Request lifecycle



Other end states: Rejected, Cancelled, Expired and No Capacity Available.

Action	Who	Effect on capacity
Send request	Patient / coordinator	available -1, reserved +1 (per resource). Expiry set to now + 15 minutes.
Accept	Staff of that hospital	reserved -1, occupied +1. Status accepted. accepted_at set. Audit log written.
Reject	Staff of that hospital	reserved -1, available +1. Status rejected.
Cancel	The requester	reserved -1, available +1. Status cancelled.
Expire	Background job	reserved -1, available +1. Status expired.
Mark transferred / admitted	Staff	No capacity change. Status moves forward in order only.

Invalid transitions are blocked, for example accepting an expired or already rejected request, or jumping from accepted straight to admitted.

8.5 The expiry job

A background task starts with the server and runs every 30 seconds. It finds requests still in request_sent or hospital_reviewing whose reservation_expires_at has passed, releases the reserved capacity exactly once and marks them expired. This means a hospital that ignores a request does not lose a bed forever.

8.6 Stale data warning

Every capacity record carries last_updated. The API returns minutes_since_update and a stale flag that is true after 30 minutes. The UI shows 'Last updated X minutes ago' and, when stale, the red warning 'Capacity information may be outdated'. The admin dashboard lists hospitals with stale data.

9. API reference

Interactive documentation is generated automatically at <http://localhost:8000/docs> when the backend is running. Every endpoint except /health and /auth/login requires the header **Authorization: Bearer TOKEN**.

Method and path	Role	Purpose
GET /health	public	Server status check.
POST /auth/login	public	Email + password. Returns access_token, role, name, hospital_id.
GET /auth/me	any	Current user details.
GET /capacity/my-hospital	staff	All capacity records of the staff user's hospital.
PUT /capacity/{resource_type}	staff	Update total, occupied, temporarily_unavailable. Recalculates available, stamps last_updated, writes audit log.
GET /hospitals/{id}/capacity	any	Capacity of one hospital with minutes_since_update and stale flag.
POST /search	any	Find, filter and rank hospitals. Returns suitable and excluded lists.
POST /requests	patient, coordinator	Create a referral and reserve capacity atomically.
GET /requests/mine	patient, coordinator	Own requests with status and minutes left.
GET /requests/hospital	staff	Requests for the staff user's hospital, newest first.
PATCH /requests/{id}/accept	staff (that hospital)	Reserved becomes occupied. Status accepted.
PATCH /requests/{id}/reject	staff (that hospital)	Release reservation. Status rejected.
PATCH /requests/{id}/cancel	requester	Release reservation. Status cancelled.
PATCH /requests/{id}/status	staff	Move accepted to patient_transferred to admitted.
GET /analytics/hospital	staff	Totals, occupancy per resource, ICU occupancy, requests by status, average response time, last 7 days.
GET /analytics/system	admin	Top occupied hospitals, most requested resources, requests by status and urgency, average response time, stale hospitals.
GET /admin/hospitals	admin	All hospitals with status, last update, coordinates and occupancy.
PATCH /admin/hospitals/{id}/verify	admin	Set verification_status to verified or rejected.

9.1 Example: search

Request

```
POST /search
{
  "required_resources": ["icu_bed", "ventilator"],
  "latitude": 25.3960,
  "longitude": 68.3578,
  "max_distance_km": 30
}
```

Response (shortened)

```
{
```

```

"suitable": [
  { "hospital_id": "...", "name": "Autobahn Road Hospital", "distance_km": 3.4,
    "estimated_travel_minutes": 7, "match_percent": 88,
    "capacity": { "icu_bed": { "available": 8, "total": 18},
                  "ventilator": { "available": 6, "total": 10} },
    "last_updated": "2026-10-05T09:12:00Z", "stale": false }
],
"excluded": [
  { "name": "Hirabad Medical Centre", "reason": "No ventilator available" },
  { "name": "Saddar City Hospital", "reason": "No ICU bed available" }
]
}

```

9.2 Error codes you will see

Code	Meaning
400	Invalid capacity numbers (negative or more than total).
401	Missing, expired or tampered token. The app logs the user out.
403	Role or hospital not allowed for this action.
409	No capacity available (bed just taken) or invalid status transition.
422	Request body failed validation (wrong type, missing field, unknown resource type).

10. The Flutter app

10.1 Screens by role

Role	Screen	What it does
All	Login	Email and password, role chips that fill demo credentials, session restored on refresh.
Patient / coordinator	Find Hospital	Pick resources, location (presets, typed or GPS), max distance, urgency. Shows ranked results with match ring, distance, travel time, availability, and a Not suitable section.
Patient / coordinator	Results map	List or Map toggle. Tap a pin for details, Send Request or Navigate.
Patient / coordinator	My Requests	Status timeline, reservation countdown, cancel button, mini map and Navigate button once accepted. Refreshes every 4 seconds.
Staff	Capacity	One card per resource with occupancy bar and last-updated label. Edit dialog with validation.
Staff	Requests	Incoming requests sorted by urgency, countdown, Accept, Reject, Transferred, Admitted, patient location button. Refreshes every 4 seconds.
Staff	Dashboard	Stat tiles, occupancy bar chart, request status pie chart, 7-day line chart.
Admin	System Analytics	Totals, top 5 occupied hospitals, most requested resources, trend, stale list.
Admin	Hospitals	All hospitals with status chips, Verify and Reject buttons.
Admin	Map	All hospitals as pins coloured by occupancy.

10.2 Session and navigation

auth_provider stores the token, role, name and hospital_id in shared_preferences, restores them on start and routes to PatientHome, StaffHome or AdminHome. api_client adds the Bearer token to every request and, on a 401 response, logs the user out cleanly.

10.3 Design system

The UI uses a red, black and white identity with Material 3. All colors, fonts and component styles are defined once in **config/app_theme.dart** and reusable widgets in **widgets/**, so a restyle means editing one place.

Token	Value	Used for
Primary red	#E11D2E (dark #B0121F, tint #FDECEE)	Buttons, active nav, pins, accents
Black / ink	#0B0B0F / #16161D	Sidebar, headers, text
White / page	#FFFFFF / #F5F5F7	Cards and page background
Borders / secondary text	#E5E7EB / #6B7280	Card borders, captions
Status green / amber	#16A34A / #F59E0B	Availability dots and badges only
Fonts	Inter (body), Sora (headings, big numbers)	Via google_fonts

Layout is responsive: at 900 px wide or more there is a black left sidebar, below that a bottom navigation bar. Reusable widgets: AppButton, AppCard, StatTile, StatusChip, SectionHeader, EmptyState, LoadingSkeleton, MatchRing and HospitalMap.

10.4 Map system

- Tiles come from OpenStreetMap through flutter_map. No API key or billing is required.
- Hospital pins are red; the best match is larger with a #1 label; excluded hospitals are grey.
- 'Use my location' fills coordinates from GPS. If permission is denied, the presets still work.
- 'Navigate' opens <https://www.google.com/maps/dir/?api=1&destination=LAT,LNG> in Google Maps, which gives live turn-by-turn directions.
- GPS in the browser only works on HTTPS or localhost, so it works after deployment but not on a plain HTTP page.

11. Demo data and logins

Run seed.py to create the data below. Capacity shown is total / occupied. Other resource types (operation theatre, isolation, dialysis, trauma, ambulance) are generated per hospital.

#	Hospital	Area	ICU	Ventilator	Emergency	Notes
1	Sindh Care Teaching Hospital	Qasimabad	20/15	12/8	30/22	
2	Latifabad Civil Hospital	Latifabad Unit 7	15/12	8/5	25/20	
3	Hirabad Medical Centre	Hirabad	10/6	4/4	15/9	Trap: no ventilators free
4	Saddar City Hospital	Saddar	12/12	6/6	20/18	Trap: ICU full
5	Autobahn Road Hospital	Autobahn Road	18/10	10/4	28/15	
6	Gulistan-e-Sajjad Hospital	Gulistan-e-Sajjad	8/5	5/2	12/8	
7	Jamshoro Road Teaching Hospital	Jamshoro Road	25/18	15/9	35/25	Largest
8	Fateh Chowk Emergency Hospital	Fateh Chowk	6/5	4/3	40/34	High trauma
9	Kotri Road Medical Centre	Kotri Road	10/4	6/2	14/6	
10	Pretabad Women and Children Hospital	Pretabad	6/3	3/1	10/5	Large NICU, no trauma

Demo users: 1 admin, 1 coordinator, 1 patient and 1 staff user per hospital (13 users). All demo passwords are **demo123**. The seed script prints the full list of emails and roles when it finishes, and the README lists them too.

Location presets: City Centre (25.3960, 68.3578), Qasimabad, Latifabad, Autobahn Road, Gulistan-e-Sajjad, Kotri Road, Jamshoro Road.

seed_requests.py adds about 40 past requests spread over the last 7 days so charts have data.
reset_demo.py re-runs both scripts to return to a clean state before recording a demo.

Note. Seed scripts clear collections before inserting. Never run them against a database holding real data.

12. Running the project locally

12.1 Prerequisites

- Python 3.10 or newer, Flutter SDK (with Chrome for web), Git.
- A free MongoDB Atlas cluster, a database user, network access set to allow your IP (or 0.0.0.0/0 for development) and the connection string.

12.2 Backend

```
cd backend
python -m venv venv
source venv/bin/activate           # Windows: venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env              # then fill in MONGODB_URI and JWT_SECRET
python seed.py                    # hospitals, capacities, users
python seed_requests.py           # fake history for charts
uvicorn main:app --reload --port 8000
```

Open <http://localhost:8000/health> (expect status ok) and <http://localhost:8000/docs> for the interactive API.

12.3 Frontend

```
cd frontend
flutter pub get
flutter run -d chrome
```

The app calls <http://localhost:8000> by default. Keep the backend running while you use the app.

12.4 Quick smoke test

- Log in as the staff demo user and edit ICU numbers. Refresh and confirm they persist.
- In a second browser window log in as the patient, search ICU + ventilator, send a request.
- Back in the staff window the request appears within 5 seconds. Accept it and watch the numbers change.

13. Configuration and environment variables

Variable	Where	Purpose
MONGODB_URI	backend/.env	MongoDB Atlas connection string.
JWT_SECRET	backend/.env	Long random string used to sign tokens. Keep private, change per environment.
ALLOWED_ORIGINS	backend env	Comma-separated list of allowed web origins for CORS. Default is *. Set to your deployed web URL in production.
PORT	hosting platform	Port the server listens on (set by Render or Railway).
API_BASE_URL	Flutter --dart-define	Backend URL baked into a build. Falls back to localhost:8000.

Never commit .env. The .gitignore excludes it along with venv, build output, .dart_tool and __pycache__.

14. Deployment guide

14.1 Backend (Render or Railway)

- Push the repo to GitHub and create a new Web Service pointing at the backend folder.
- Build command: `pip install -r requirements.txt`
- Start command: `uvicorn main:app --host 0.0.0.0 --port $PORT`
- Add environment variables MONGODB_URI, JWT_SECRET and ALLOWED_ORIGINS.
- In Atlas, make sure network access allows the host (0.0.0.0/0 is simplest for a hackathon).
- Run `python seed.py` once against the production database (from your machine with the production URI) to create demo data.
- Open `https://YOUR-BACKEND/health` to confirm.

14.2 Web app (Vercel or Firebase Hosting)

```
cd frontend
flutter build web --release --dart-define=API_BASE_URL=https://YOUR-BACKEND
# upload the build/web folder to Vercel or Firebase Hosting
```

After deploy, set ALLOWED_ORIGINS on the backend to the web URL, then redeploy the backend.

14.3 Android APK

```
flutter build apk --release --dart-define=API_BASE_URL=https://YOUR-BACKEND
# output: build/app/outputs/flutter-apk/app-release.apk
```

14.4 Pre-deploy checklist

- `flutter analyze` shows no issues; `flutter build web` and `flutter build apk` succeed.
- No secrets in the repository; `.env.example` lists variable names only.
- Requirements are pinned and install in a fresh virtual environment.
- All role and permission tests pass (see section 16).
- `reset_demo.py` leaves the app in a clean demo state.

Tip. Free-tier backends sleep after inactivity and the first request can take 30 to 60 seconds. Open the `/health` URL a minute before any live demo or video recording.

15. Security design

- **Passwords:** stored only as bcrypt hashes. The hash is never included in API responses.
- **Tokens:** JWT signed with JWT_SECRET, 24-hour expiry, carrying user_id, role and hospital_id. Tampered or expired tokens are rejected with 401.
- **Role enforcement:** a require_role dependency returns 403 when the role is not allowed.
- **Hospital isolation:** staff are tied to one hospital through the token. They cannot edit capacity or act on requests belonging to another hospital.
- **Request ownership:** a user can view and cancel only their own requests.
- **Validation:** Pydantic models reject wrong types, unknown resource types and invalid urgencies with 422.
- **Audit trail:** every capacity change is logged with old and new values.
- **Secrets:** database URI and JWT secret live in environment variables, not in code.
- **Database access:** only the backend holds credentials. The app never connects to MongoDB.

15.1 Before using this with real data

This is a hackathon-grade system. Before any real-world use you would need, at minimum: HTTPS everywhere, a restricted Atlas IP allow-list, rate limiting on login, refresh tokens or shorter expiry, proper hospital onboarding and verification, privacy review of patient data and compliance with local health-data law.

16. Testing and quality checks

Area	What to verify
Backend functional	Login for all roles, capacity update and validation, search ranking and exclusions, full request lifecycle, analytics numbers match raw data.
Concurrency	Ten simultaneous requests for a resource with one bed left: exactly one succeeds, nine get 409.
Data integrity	For every capacity record available equals total minus occupied, reserved and unavailable, and nothing is negative. Expiry releases capacity exactly once.
Security	Role matrix per endpoint (200, 401, 403), cross-hospital access blocked, no password hash in responses, tampered token rejected, no stack traces to the client.
Frontend	flutter analyze clean, builds succeed, all four role logins, session survives refresh, full two-window loop, empty and error states, no layout overflow at 360 px and 1280 px widths.
Deployment	Fresh install from requirements.txt starts the server, environment-driven CORS and API URL, indexes created on an empty database.

17. How to modify and extend

17.1 Change the city or hospitals

Edit the hospital list in backend/seed.py (name, address, latitude, longitude, capacities) and run it again. Update the location presets and default location in the Flutter app config. Remember coordinates are stored as [longitude, latitude].

17.2 Add a new resource type

- Add it to the resource type list in the backend model (and the validation enum).
- Add capacity records for it in seed.py.
- Add a label and icon for it in the Flutter resource selector and capacity card.

17.3 Tune the matching score

Open `services/matcher.py` and change the weights (0.5, 0.3, 0.2). They should add up to 1. To favour closeness more, increase the distance weight. The average speed used for travel time (30 km/h) is in the same file.

17.4 Change time limits

- Reservation length (15 minutes): in `services/referral.py` where `reservation_expires_at` is set.
- Expiry job interval (30 seconds): where the background task is started in `main.py`.
- Stale threshold (30 minutes): in the capacity and search services, and the matching UI warning.
- Polling intervals (4 s and 10 s): the Timer durations in the Flutter screens.
- Token lifetime (24 hours): `services/auth.py`.

17.5 Restyle the app

Change colors, fonts, radii and button heights in `config/app_theme.dart`. Components pick the changes up automatically because they use theme tokens.

17.6 Add a new API endpoint

- Create the route function in the matching file under `routes/` and protect it with `require_role`.
- Put the logic in a function under `services/` so it can be tested alone.
- Register the router in `main.py` if you created a new file.
- Call it from a Flutter service class and show it in a screen.

17.7 Add a new role

Add the role to the User model, create a dependency with `require_role('newrole')` on the endpoints it may use, add seed users, and add a new Home screen and route in the Flutter role router.

17.8 Ideas for future work

- AI request classifier: free-text case description turned into resources and urgency (planned and skipped for now).
- Capacity forecasting and demand prediction from request history.
- Push or SMS notifications when a request is accepted.
- WebSockets instead of polling.
- Real road routing for travel time instead of distance divided by speed.
- QR-based referral confirmation, ambulance tracking, regional heatmap.
- Automated tests and a CI pipeline.

18. Troubleshooting

Problem	Likely cause and fix
Backend says MongoDB not connected	Wrong MONGODB_URI, password with special characters not URL-encoded, or your IP is not allowed in Atlas network access.
App shows network error or blank data	Backend not running, wrong API_BASE_URL, or CORS blocking the web origin. Set ALLOWED_ORIGINS to include your web URL.
Everything returns 401	Token expired or JWT_SECRET changed. Log in again.
Search returns no hospitals	Seed data missing, hospitals not verified and active, or max distance too small. Also check coordinates are [longitude, latitude].
Request returns 409	The bed was just taken or the status change is not allowed from the current state.
Charts are empty	Run seed_requests.py.
GPS button does nothing on the web	Browsers allow location only on HTTPS or localhost, and the user must grant permission.
Map tiles do not load	No internet access to tile.openstreetmap.org.
Deployed backend very slow at first	Free tier is waking up. Wait, then retry.
Flutter overflow stripes	A layout does not fit the width. Check the screen at 360 px and wrap in Expanded or scrollable widgets.

19. Limitations

- Live updates use polling, so changes can take a few seconds to appear.
- Travel time is estimated from straight-line distance and a fixed speed, not real roads or traffic.
- Demo data is fictional. Capacity is only as accurate as what staff enter, which is why timestamps and stale-data warnings exist.
- Reservation expiry runs inside the server process; with multiple server instances each would run it. Atomic updates keep this safe, but a dedicated scheduler is cleaner at scale.
- No email, SMS or push notifications yet.
- Not yet reviewed for medical-data regulations.

20. Glossary

Term	Meaning
Atomic operation	A database change that happens completely or not at all, with no other change able to slip in between its check and its update.
CORS	Browser rule that decides which websites may call the backend.
Dio	Flutter HTTP client library.
FastAPI	Python framework for building APIs.
GeoJSON / 2dsphere	Standard location format and the MongoDB index that makes distance queries fast.
JWT	JSON Web Token: a signed token proving who the user is and their role.
Match percentage	0 to 100 score combining distance, availability and low occupancy.
Polling	The app asking the server for fresh data every few seconds.
Provider	Flutter state management package.
Reservation	Temporary hold on a bed (reserved count) while a request is pending.
Seed data	Fake starting data inserted for demos and testing.
Stale data	Capacity numbers not updated for over 30 minutes.